

PhaseServe: Efficient Phase-Disaggregated LLM Serving with Phase-Specialized Coordinated Scheduling

Jie Dai, Meng Hao, Weizhe Zhang, Senior Member, IEEE, Jiaxi Li, Hongwei Yang, Hui He, and Desheng Wang

Abstract—Large language model (LLM) inference has become increasingly important across a wide range of AI applications, yet efficient serving remains challenging due to the two-phase nature of LLM execution: prefill and decoding. These two phases exhibit fundamentally different execution characteristics. Prefill is compute-intensive and its cost is largely determined by input length, whereas decoding is memory-bandwidth-intensive and its total cost is only revealed incrementally during generation. Although phase-disaggregated architectures reduce interference between the two phases, they still suffer from significant intra-phase inefficiencies under heterogeneous workloads. In this work, we present PhaseServe, a phase-disaggregated LLM serving system built on phase-specialized scheduling. For the prefill phase, PhaseServe combines length-aware scheduling with capacity-aware batch construction to improve workload matching and reduce latency under heterogeneous prompt lengths. For decoding, it employs an MLFQ-based attained-service-aware scheduler with proactive KV-cache management to improve token-level responsiveness under variable output lengths. Experimental results show that, under the tested deployment setting, PhaseServe improves both time to first token (TTFT) and time per output token (TPOT) compared with representative baselines, achieving up to $9\times$ speedup in TTFT and $2\text{--}2.5\times$ speedup in TPOT.

Index Terms—Large Language Models, inference systems, prefill-decode optimization, dynamic scheduling, batching, KV-cache management.

I. Introduction

LARGE Language Models (LLMs), such as LLaMA [1], GPT [2], and OPT [3], are rapidly becoming the serving backend of interactive AI applications, including conversational assistants [4], [5], code-generation systems [6], [7], and enterprise copilots. Unlike conventional inference services, LLM serving must sustain high throughput while simultaneously meeting strict latency objectives under highly variable request mixes. In practice, users care about both time to first token (TTFT), which determines how quickly the system starts responding, and time per output token (TPOT), which determines how responsive generation remains once decoding begins. Meeting both targets efficiently is challenging because LLM inference

is expensive in both computation and memory, and real workloads exhibit highly skewed input and output lengths.

Modern LLM serving systems have substantially improved performance through continuous batching, memory-efficient KV-cache management, and distributed execution [8]–[15]. However, these systems still face a fundamental tension. Prefill and decode have very different execution characteristics: prefill processes the full prompt and is predominantly compute-intensive, whereas decode generates tokens iteratively and is typically constrained by memory traffic and token-level queuing delay. When the two phases are scheduled together, long-running prefills can delay latency-sensitive decode iterations, making it difficult to satisfy TTFT and TPOT simultaneously.

Recent phase-disaggregated (PD) systems address this problem by separating prefill and decode onto different instances [15]–[17]. This is an important architectural step: it removes direct interference between the two phases and enables phase-specific provisioning. However, phase disaggregation does not eliminate all inefficiency. Instead, it exposes a new bottleneck that existing systems do not explicitly address: intra-phase scheduling inefficiency under heterogeneous workloads. In particular, prefill requests exhibit highly skewed prompt lengths and therefore highly skewed execution costs, whereas decode requests have unknown and dynamically revealed output lengths, which lead to stragglers and inflated token-level latency.

The key insight of this paper is that these two inefficiencies correspond to different scheduling regimes. In prefill, the dominant cost of a request is largely determined by its prompt length and is therefore known before execution. Prefill is thus naturally a scheduling problem with known but heterogeneous job sizes. In decode, the total service demand is not known a priori and only becomes visible as generation proceeds. Decode is therefore an iterative scheduling problem with unknown and dynamically evolving job sizes. This asymmetry implies that a unified policy is fundamentally mismatched to PD serving. Efficient phase-disaggregated serving requires phase-specialized scheduling: prefill should exploit size information to reduce head-of-line blocking and improve batching quality, while decode should use service-aware iterative scheduling to control stragglers and token-level latency.

Based on this insight, we present PhaseServe, a

Jie Dai, Meng Hao, Weizhe Zhang, Jiaxi Li, Hongwei Yang, and Hui He are with the Harbin Institute of Technology, China. Desheng Wang is with the Harbin Institute of Technology (Shenzhen), China.
E-mail: daijie@stu.hit.edu.cn, haomeng@hit.edu.cn, wz-zhang@hit.edu.cn, lijiaxi040601@163.com, yanghongwei@hit.edu.cn, hehui@hit.edu.cn, wangdesheng@hit.edu.cn.

Corresponding authors: Meng Hao, Weizhe Zhang.

phase-disaggregated LLM serving system with coordinated, phase-specialized scheduling. PhaseServe combines a lightweight global controller for cross-phase routing and load balancing with two different local scheduling policies. On the prefill side, PhaseServe performs utility-aware batch construction that exploits known prompt costs to reduce head-of-line blocking while preserving high GPU utilization. On the decode side, PhaseServe performs service-aware iterative scheduling with KV-state-aware admission and proactive state management, enabling responsive handling of requests with unknown output lengths under explicit memory constraints. Together, these mechanisms turn phase disaggregation from a static architectural split into a coordinated serving framework that aligns scheduling policy with the information structure of each phase.

We evaluate PhaseServe on representative dialogue and code-generation workloads using OPT models [3], [6], [18]. Across a range of concurrency levels and workload mixes, PhaseServe consistently improves both TTFT and TPOT over representative unified and phase-disaggregated baselines. Under high concurrency, it achieves up to $9\times$ lower TTFT and up to $2.5\times$ lower TPOT.

In summary, this paper makes the following contributions:

- We identify intra-phase scheduling inefficiency as a fundamental bottleneck of phase-disaggregated LLM serving and show that it arises from the mismatch between heterogeneous request distributions and uniform scheduling policies.
- We formulate a phase-specialized scheduling principle for PD serving: prefill corresponds to scheduling with known job sizes, while decode corresponds to iterative scheduling with unknown and dynamically revealed job sizes.
- We design and implement PhaseServe, a coordinated serving system that integrates utility-aware prefill scheduling, service-aware decode scheduling, and KV-state-aware execution under a phase-disaggregated architecture.
- We show experimentally that PhaseServe substantially improves TTFT, TPOT, and overall serving efficiency across representative interactive workloads.

Artifact Availability. The PhaseServe implementation is publicly available at <https://github.com/Dreamer-HIT/PhaseServe>.

II. Background and Motivation

This section motivates PhaseServe from a systems perspective. We first summarize the execution characteristics of generative LLM inference and explain why TTFT and TPOT are governed by different bottlenecks (Section II-A). We then discuss why existing optimizations, although important, are insufficient for heterogeneous phase-disaggregated serving (Section II-B). Finally, we present empirical evidence showing that the two phases exhibit distinct internal inefficiencies and should therefore be treated as different scheduling problems (Section II-C).

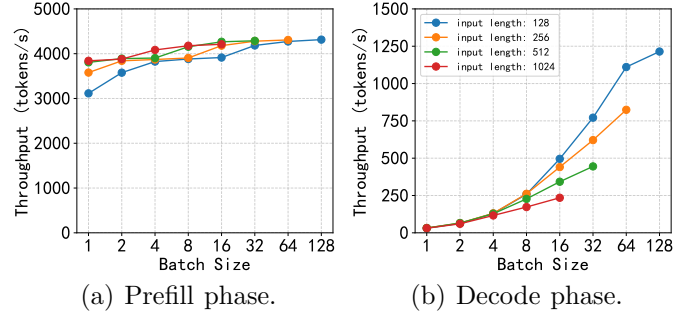


Fig. 1. Throughput of the two inference phases under different batch sizes and sequence lengths (LLaMA-3.1 8B on a single RTX 3090 GPU). Prefill saturates once prompt processing reaches the device’s compute limit, whereas decode continues to scale with batch size until memory bandwidth becomes the bottleneck.

A. Generative LLM Inference

Decoder-only LLMs [1]–[3], [19] generate text autoregressively. From a systems perspective, inference naturally decomposes into two sequential phases: prefill and decode.

Prefill. In the prefill phase, the model processes the entire input prompt, computes hidden states for all input tokens, constructs the initial KV cache, and produces the first output token. Since all prompt tokens are processed together across Transformer layers, prefill exposes substantial matrix-computation parallelism and is predominantly compute-intensive. Its dominant cost is strongly correlated with input length, which makes prefill cost largely predictable before execution.

Decode. In the decode phase, the model generates one token per iteration. Although each step introduces only one new token, attention must read the accumulated KV cache from all prior tokens. As a result, decode performs relatively little new computation per step but incurs substantial memory traffic, especially under concurrency. Decode is therefore typically more sensitive to memory bandwidth and token-level queuing delay than raw compute throughput.

These differences imply that TTFT and TPOT are governed by different resource bottlenecks. TTFT is dominated by prefill execution and prefill-side queueing delay, whereas TPOT is dominated by decode-side iteration latency and KV-cache pressure.

Figure 1 illustrates this asymmetry empirically. As prompt length grows, prefill throughput eventually plateaus, indicating that the device has entered a compute-saturated regime. In contrast, decode throughput continues to improve as batch size increases until memory bandwidth becomes the limiting factor. This distinction is central to our design: prefill and decode are not merely two steps of the same pipeline, but two workload classes with different bottlenecks and different scheduling signals.

B. Existing Optimizations and Their Limits

Modern LLM serving systems rely on batching, memory management, and phase separation to improve performance. These techniques are essential, but none of them

fully address the scheduling mismatch exposed by heterogeneous phase-disaggregated workloads.

Batching and continuous scheduling. Batching amortizes model execution cost across requests and is therefore a primary mechanism for improving throughput. However, naive static batching is ineffective for LLMs because input and output lengths vary widely, causing severe straggler effects. Systems such as Orca [8], vLLM [9], and TensorRT-LLM [12] therefore adopt continuous batching, allowing requests to join and leave a running batch dynamically. While this substantially improves utilization, it still does not resolve the fundamental mismatch between prefill and decode when both are scheduled within the same domain.

KV-cache memory management. The KV cache grows with sequence length and can occupy a large fraction of GPU memory, directly limiting concurrency. vLLM [9] addresses this issue with PagedAttention, which allocates KV state incrementally in fixed-size blocks rather than reserving large contiguous buffers upfront. This design improves memory efficiency and batching capacity. Nevertheless, better KV allocation alone does not determine which requests should be served first, nor does it address the phase-specific nature of prefill and decode scheduling.

Phase-disaggregated architectures. Recent systems such as Splitwise [16], DéjàVu [17], and DistServe [15] separate prefill and decode onto different instances. This removes direct phase interference and enables phase-specific resource provisioning. However, once the two phases are separated, each phase still faces its own internal scheduling bottleneck. Prefill remains vulnerable to batching inefficiency caused by highly skewed prompt lengths, while decode remains vulnerable to straggler effects caused by unknown generation lengths.

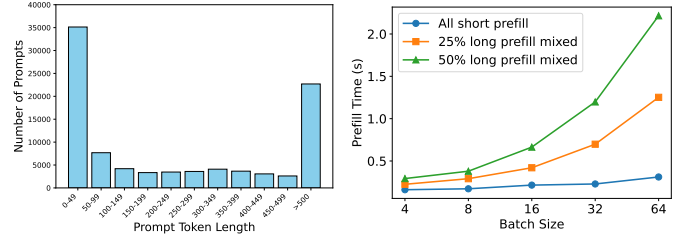
The limitation of existing approaches is therefore not the absence of batching, memory optimization, or disaggregation per se. Rather, it is the absence of a phase-specialized scheduling framework that explicitly matches the scheduling policy to the information structure of each phase.

C. Problems and Opportunities

We now empirically characterize the inefficiencies that persist even after phase disaggregation. Our findings show that the two phases exhibit different failure modes and should be treated as different scheduling problems.

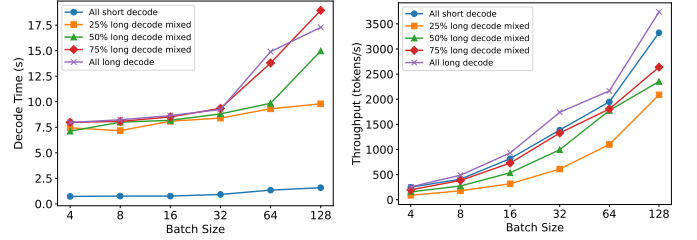
Observation 1: Prefill is a scheduling problem with known but highly skewed job sizes. Prompt lengths in real workloads vary substantially, and this variation directly determines prefill cost. Figure 2a shows the prompt-length distribution of ShareGPT, which is highly skewed: short prompts are common, but long prompts remain frequent enough to create substantial heterogeneity. This workload shape implies that prefill naturally corresponds to a scheduling setting in which job sizes are largely known but highly imbalanced.

We further measure the impact of mixing short and long prompts in the same prefill batch. Figure 2b shows that



(a) Prompt length distribution in ShareGPT. (b) Prefill latency under mixed prompt lengths.

Fig. 2. Empirical evidence of prefill-side heterogeneity. Real workloads exhibit highly skewed prompt lengths, and mixing short and long prompts in the same batch significantly inflates prefill latency.



(a) Decode latency under mixed output lengths. (b) Decode throughput under mixed output lengths.

Fig. 3. Empirical evidence of decode-side heterogeneity. Long generations remain in the system for many more iterations, causing straggler effects that increase latency and reduce throughput for short requests.

when only a fraction of long prompts is introduced into an otherwise short-prompt batch, overall prefill latency increases sharply. The reason is that prefill execution time is dominated by the longest sequence in the batch. Shorter requests are therefore forced to wait behind longer ones, inflating TTFT and reducing effective compute utilization. Importantly, this inefficiency is not eliminated by continuous batching or by phase disaggregation itself, because it is rooted in the mismatch between skewed prompt costs and batch-level execution semantics.

Observation 2: Decode is a scheduling problem with unknown and dynamically revealed job sizes. Decode exhibits a fundamentally different structure. While the cost of each individual iteration is relatively stable, the total number of iterations required by a request is unknown a priori and varies significantly across requests. Figure 3 shows that mixing short and long generations in the same decode workload significantly increases latency and reduces throughput. Long-running requests remain active for many more iterations, thereby occupying decode slots and delaying the completion of short requests.

This behavior is a classic straggler phenomenon. Unlike prefill, however, it cannot be effectively addressed by simple size-based grouping or static batching, because the total generation length is not known in advance. Instead, the scheduler must make decisions online based on attained service and evolving request state.

Design implication. These observations reveal that the

bottlenecks of phase-disaggregated serving are not uniform. Prefill corresponds to a scheduling regime with known job sizes and highly skewed cost distribution, suggesting size-aware batching and bounded-wait prioritization. Decode corresponds to an iterative scheduling regime with unknown job sizes, suggesting service-aware scheduling with explicit fairness and state management. Therefore, an efficient PD serving system should not apply a single generic policy to both phases. Instead, it should explicitly align each phase with the scheduling signals and control mechanisms appropriate to its workload structure.

This insight motivates the design of PhaseServe. Rather than treating phase disaggregation as the end of optimization, PhaseServe treats it as the beginning of a new scheduling regime: once prefill and decode are separated, each phase should be scheduled according to its own information structure, bottleneck, and latency objective.

III. PhaseServe Design

A. Overview

PhaseServe is a phase-disaggregated LLM serving system that targets SLO-constrained goodput: it seeks to maximize the sustainable request rate while satisfying both TTFT and TPOT targets for the vast majority of requests. The design is motivated by a simple but important observation: after prefill and decode are disaggregated, the main bottlenecks no longer come only from direct phase interference, but also from intra-phase heterogeneity and cross-phase imbalance. Prefill requests have known but highly heterogeneous execution costs before execution, whereas decode requests reveal their total service demand only incrementally as tokens are generated. As a result, the two phases require fundamentally different local scheduling policies, and the system as a whole requires explicit cross-phase control.

$$\max \text{Goodput} \quad \text{s.t.} \quad \text{TTFT}_{p90} \leq \tau_{\text{TTFT}}, \text{TPOT}_{p90} \leq \tau_{\text{TPOT}} \quad (1)$$

Figure 4 shows the architecture of PhaseServe. The system consists of four major components: a global phase controller, a pool of prefill engines, a pool of decode engines, and a KV-state manager. Requests first enter the global ingress queue and are routed to a prefill engine. After prefill completes and the initial KV state is materialized, the request is handed off to a decode engine for autoregressive generation. Throughout execution, the KV-state manager tracks the residency and transfer status of request state and exposes this information to both the global controller and local schedulers.

PhaseServe is organized as a hierarchical control system. The global phase controller performs coarse-grained decisions on routing, phase balancing, and control-epoch adaptation. Each prefill engine performs utility-aware batch construction for known-size jobs, and each decode engine performs service-aware iterative scheduling for unknown-size jobs under explicit KV and prefetch budgets. This

Placeholder for System Overview Figure

Recommended drawing: a full pipeline with four blocks: Global Phase Controller → Prefill Engines → KV-State Manager → Decode Engines.

Show request arrival at the controller, routing to one prefill engine, prefill completion and KV registration, topology-aware handoff to one decode engine, and iterative token generation.

Add feedback arrows from prefill/decode pools back to the controller labeled: prefill backlog, decode backlog, TTFT/TPOT miss rate, and KV-transfer pressure.

Inside the controller, annotate four functions: load estimation, routing, phase-balance tuning, and policy adaptation.

Fig. 4. Architecture of PhaseServe. A global phase controller performs routing and closed-loop phase balancing. Prefill engines use utility-aware batch construction for known-size requests. Decode engines use service-aware iterative scheduling under KV-state control. The KV-state manager explicitly tracks residency, prefetch, eviction, and handoff.

organization separates long-timescale control from short-timescale dispatch, allowing PhaseServe to react to both dynamic workload composition and iteration-level service variability.

B. Global Phase Controller

The global phase controller is the control plane of PhaseServe. It maintains a lightweight but continuously updated view of system state, including the pending workload of each prefill engine, the active decode backlog of each decode engine, KV-transfer pressure, and observed SLO violations. The controller is invoked on request arrival and at every control epoch.

1) Request Characterization and Cost Proxies: For each request r , the controller extracts a compact descriptor

$$\phi(r) = \langle l_{\text{in}}, \hat{l}_{\text{out}}, s, c \rangle,$$

where l_{in} is the input length, $\hat{l}_{\text{out}}$ is an optional coarse output-length class, s denotes the request or SLO class, and c denotes the current context length once the request enters decode.

PhaseServe does not require highly accurate latency prediction. Instead, it relies on robust phase-specific cost proxies that are sufficient for online control. For prefill, the request cost is approximated as

$$\hat{C}_{\text{prefill}}(r) = a_1 l_{\text{in}} + a_2 l_{\text{in}}^2, \quad (2)$$

which captures the dominant scaling behavior of prompt processing with respect to input length. For decode, the instantaneous cost is approximated as

$$\hat{C}_{\text{decode}}(r) = b_1 c + b_2 m_{\text{kv}}(r), \quad (3)$$

where $m_{\text{kv}}(r)$ denotes the KV-cache footprint of request r . This proxy reflects both context growth and decode-side memory pressure.

2) Phase-Specific Load Estimation: Using these proxies, the controller maintains phase-specific load estimates. For a prefill engine e_p , the estimated load is

$$\hat{L}_p(e_p) = \sum_{r \in Q_p(e_p)} \hat{C}_{\text{prefill}}(r), \quad (4)$$

where $Q_p(e_p)$ includes both queued and currently executing prefill requests. For a decode engine e_d , the estimated load is

$$\hat{L}_d(e_d) = \sum_{r \in A(e_d)} \hat{C}_{\text{decode}}(r), \quad (5)$$

where $A(e_d)$ is the active decode set.

This separation is important. Prefill pressure is dominated by known prompt-processing cost, whereas decode pressure is dominated by iterative context growth, KV footprint, and activation cost. A single aggregated load signal would obscure this asymmetry and lead to unstable routing.

3) Routing and Topology-Aware Handoff: Upon arrival, a request is routed to the prefill engine minimizing

$$\text{Score}_p(e_p, r) = w_1 \hat{L}_p(e_p) + w_2 \hat{Q}_p(e_p) + w_3 \text{SLOPenalty}(e_p, r), \quad (6)$$

where $\hat{Q}_p(e_p)$ denotes queueing pressure and $\text{SLOPenalty}(e_p, r)$ estimates the TTFT risk for request r on engine e_p .

After prefill completes, the request is forwarded to the decode engine minimizing

$$\text{Score}_d(e_d, r) = u_1 \hat{L}_d(e_d) + u_2 T_{\text{kv}}(e_p, e_d, r) + u_3 \text{NetCost}(e_p, e_d), \quad (7)$$

where $T_{\text{kv}}(e_p, e_d, r)$ estimates the KV-state handoff cost from prefill engine e_p to decode engine e_d , and $\text{NetCost}(e_p, e_d)$ captures topology-dependent communication cost. This formulation makes routing explicitly state-aware: decode placement is treated as a joint scheduling-and-handoff decision rather than a pure queue-balancing problem.

4) Closed-Loop Phase Balancing: Phase disaggregation removes direct prefill–decode interference, but also introduces a producer–consumer relation between the two phases. Under realistic workloads, prefill may over-produce requests relative to decode capacity, or decode may be underutilized because prefill has become the bottleneck. To address this issue, the controller operates in repeated control epochs. At the end of each epoch, it collects four signals:

- prefill backlog growth,
- decode backlog growth,
- TTFT/TPOT SLO miss rate,
- KV-transfer saturation.

Based on these signals, the controller updates three classes of parameters:

- prefill aggressiveness, including the waiting-window size and target prefill workload;
- decode aggressiveness, including queue quanta, starvation threshold, and admission conservativeness;
- cross-phase balance, including routing weights and the effective prefill/decode capacity ratio.

In effect, the controller acts as a slow-timescale optimizer layered above the local schedulers. Local engines react to instantaneous request heterogeneity, while the controller corrects structural imbalance across epochs. This separation keeps the control plane lightweight while

still allowing the system to adapt to non-stationary workloads.

C. Prefill Engine

The prefill engine executes prompt processing and first-token generation. Since the dominant cost of a prefill request is known before execution, PhaseServe models prefill scheduling as an online problem with known but highly heterogeneous job sizes. The engine consists of two tightly coupled mechanisms: windowed candidate selection and utility-aware batch construction.

1) Windowed Candidate Selection: Instead of greedily batching every newly arrived request, the prefill engine collects requests within a bounded waiting window W_p . Let $\mathcal{R}_p$ denote the set of pending requests in the local queue. At each dispatch round, the engine forms a candidate set

$$\mathcal{C}_p = \{r \in \mathcal{R}_p \mid \text{wait}(r) \leq W_p\} \cup \{r_{\text{oldest}}\}, \quad (8)$$

where r_{oldest} is always included to prevent starvation. This design avoids the pathological behavior of purely greedy batching while still bounding request waiting time.

2) Utility-Aware Batch Construction: Within $\mathcal{C}_p$, the engine constructs the next prefill batch by maximizing a phase-specific utility:

$$U_p(B) = \sum_{r \in B} \omega(r) - \lambda_1 \text{PadCost}(B) - \lambda_2 \text{LoadGap}(B) - \lambda_3 \text{MemRisk}(B), \quad (9)$$

where $\omega(r)$ is a request weight that increases with waiting age and decreases with estimated cost, $\text{PadCost}(B)$ measures inefficiency caused by batching mismatched prompt lengths together, $\text{LoadGap}(B)$ measures the distance between the estimated workload of batch B and the empirically profiled GPU saturation region, and $\text{MemRisk}(B)$ penalizes configurations likely to trigger memory pressure.

Operationally, the engine first ranks candidates using a score that combines estimated cost and age, then greedily builds a memory-feasible batch, and finally performs local refinement to improve workload matching and reduce padding waste. Compared with fixed-count batching, this formulation directly optimizes the latency–utilization tradeoff: short requests are favored in the common case, but memory constraints, batching efficiency, and fairness are all considered simultaneously.

3) Bounded-Wait Dispatch: Pure size-based policies can delay long prompts under sustained load. PhaseServe therefore introduces a dispatch timeout τ_p . If the oldest request in the candidate set has waited longer than τ_p , the engine immediately dispatches the best feasible batch even if the utility is slightly below the preferred saturation target. This yields a bounded-wait implementation of size-aware scheduling: short requests are prioritized when possible, while long requests are protected from indefinite delay.

Algorithm Implementation. Algorithm 1 implements the prefill-side candidate selection and utility-aware batch construction.

Algorithm 1 Utility-Aware Prefill Scheduling

Require: Waiting window W_p , candidate set $\mathcal{R}$, memory budget M_p , target workload L_m , dispatch timeout τ_p

Ensure: Next prefill batch B^*

```

1:  $B^* \leftarrow \emptyset$ 
2:  $\mathcal{C} \leftarrow \{r \mid r \in \mathcal{R}, \text{wait}(r) \leq W_p \text{ or } r \text{ is oldest}\}$ 
3: for all  $r \in \mathcal{C}$  do
4:   estimate  $c_r \leftarrow \hat{C}_{\text{prefill}}(r)$ 
5:   estimate  $m_r \leftarrow \hat{M}_{\text{prefill}}(r)$ 
6:   compute score  $s_r \leftarrow f(c_r, \text{wait}(r), \text{age}(r))$ 
7: end for
8: Sort  $\mathcal{C}$  by decreasing  $s_r$ 
9:  $B \leftarrow \emptyset$ 
10: for all  $r \in \mathcal{C}$  do
11:   if  $\hat{M}(B \cup \{r\}) \leq M_p$  then
12:     tentatively add  $r$  to  $B$ 
13:     if  $U_p(B \cup \{r\}) \geq U_p(B)$  then
14:        $B \leftarrow B \cup \{r\}$ 
15:     end if
16:   end if
17: end for
18: Apply local refinement on  $B$  to reduce padding waste and improve load matching
19: if  $B = \emptyset$  then
20:    $B \leftarrow \{\text{oldest request}\}$ 
21: end if
22: if  $\text{oldestWait}(\mathcal{C}) \geq \tau_p$  then
23:    $B^* \leftarrow B$ 
24: else if  $\hat{L}(B) \geq L_m$  then
25:    $B^* \leftarrow B$ 
26: end if
27: return  $B^*$ 

```

Placeholder for Prefill Scheduling Figure

Recommended drawing: compare naive FCFS/fixed-count batching with PhaseServe prefill scheduling.

Use a small set of requests with highly different prompt lengths.

Top: naive batching places short prompts behind one long prompt and creates a poorly matched batch.

Bottom: PhaseServe first forms a candidate window, then builds one or two utility-aware batches whose workload is close to the GPU saturation region.

Annotate two benefits: reduced head-of-line blocking and improved workload matching.

Fig. 5. Illustrative example of prefill scheduling under heterogeneous prompt lengths. Compared with naive batching, PhaseServe forms utility-aware batches that better match GPU saturation while reducing head-of-line blocking for short requests.

D. Decode Engine

The decode engine performs iterative token generation and directly determines TPOT. Unlike prefill, decode requests do not reveal their total service demand in advance. PhaseServe therefore models decode scheduling as an iterative service-allocation problem with unknown job sizes and explicit memory constraints. The engine combines a service-aware priority scheduler with a KV-state-aware admission mechanism.

1) Service-Aware Priority Model: Each decode request r maintains a state tuple

$$\psi(r) = \langle a(r), s(r), q(r), \delta(r) \rangle,$$

where $a(r)$ is the waiting age, $s(r)$ is the attained service measured in decode iterations, $q(r)$ is the current priority

level, and $\delta(r)$ is the TPOT slack relative to the target SLO.

PhaseServe computes a dynamic urgency score

$$P(r) = \beta_1 \frac{1}{1 + s(r)} + \beta_2 a(r) + \beta_3 \delta(r) - \beta_4 \text{PrefetchCost}(r). \quad (10)$$

This score favors lightly served requests, gradually raises the priority of starving requests, accelerates SLO-critical requests, and discounts requests whose immediate activation cost is too high. In this way, PhaseServe approximates attained-service-aware scheduling without requiring knowledge of the remaining output length.

2) Hierarchical Queue Organization: To keep scheduling overhead manageable, the priority model is implemented using a hierarchy of queues derived from MLFQ. Newly admitted decode requests enter the highest-priority queue, and long-running requests are gradually demoted as they accumulate service. However, queue transitions are not determined solely by quantum depletion. They additionally depend on starvation age, SLO slack, and KV activation readiness. As a result, the queue hierarchy acts as a practical approximation to least-attained-service scheduling rather than a blind OS-style time-sharing policy.

Let the decode scheduler maintain n queues $\{Q_1, Q_2, \dots, Q_n\}$, where Q_1 has the highest priority. The corresponding quanta satisfy

$$\tau_1 < \tau_2 < \dots < \tau_n, \quad \tau_{i+1} = 2\tau_i. \quad (11)$$

This exponential structure gives newly arrived or lightly served requests fine-grained responsiveness while amortizing scheduling overhead for long-running requests.

3) Memory-Aware Batch Admission: At each iteration, the decode engine first ranks ready requests by $P(r)$, then admits the next batch subject to memory and prefetch constraints. A request can be admitted only if

$$\text{CanAdmit}(r, B) = [m_{\text{kv}}(B \cup \{r\}) \leq M_d] \wedge [p(B \cup \{r\}) \leq P_d] \wedge [t(B \cup \{r\}) \leq T_{\text{max}}] \quad (12)$$

where M_d is the available KV budget, P_d is the prefetch bandwidth budget for the current iteration, and T_{max} is the maximum runtime token budget supported by the engine. This design ensures that a theoretically urgent request is admitted only if its KV state can be activated without destabilizing the current iteration batch.

4) Iteration Execution and Bounded Starvation: Once a batch is admitted, the engine executes exactly one decode iteration for each selected request. After the iteration, the engine updates attained service, queue level, TPOT slack, and completion status. Finished requests release their KV state immediately. Requests that exceed their local quantum or become non-urgent are demoted. Requests whose starvation timer exceeds a threshold α are promoted. By making decisions only at iteration boundaries, PhaseServe preserves responsiveness while keeping scheduling overhead practical.

Algorithm Implementation. Algorithm 2 implements the decode-side service-aware scheduling and KV-state-aware admission policy.

Algorithm 2 Service-Aware Decode Scheduling

Require: Incoming decode jobs J_{in} , ready set $\mathcal{R}$, priority queues $Q_1, \dots, Q_n$, KV budget M_d , prefetch budget P_d , max batch tokens T_{max} , starvation threshold α

Ensure: Next decode batch B^*

```

1:  $B^* \leftarrow \emptyset$ 
2: for all  $job \in J_{\text{in}}$  do
3:   initialize  $s(job) \leftarrow 0$ ,  $a(job) \leftarrow 0$ 
4:   mark KV state as resident, staged, prefetching, or remote
5:   place  $job$  into the highest-priority queue  $Q_1$ 
6: end for
7: for all  $job \in \mathcal{R}$  do
8:   update  $a(job)$ ,  $s(job)$ , and TPOT slack  $\delta(job)$ 
9:   compute  $P(job)$  according to Eq. (10)
10:  adjust queue level or in-queue order according to  $P(job)$ 
11:  if  $a(job) \geq \alpha$  then
12:    promote  $job$  to  $Q_1$ 
13:  end if
14: end for
15:  $\mathcal{C} \leftarrow$  jobs ordered by priority across  $Q_1, \dots, Q_n$ 
16: for all  $job \in \mathcal{C}$  do
17:   if CanAdmit( $job, B^*$ ) then
18:     if  $job$  is not resident then
19:       issue prefetch for  $job$ 
20:     end if
21:      $B^* \leftarrow B^* \cup \{job\}$ 
22:   end if
23: end for
24: for all  $job \in B^*$  do
25:   execute one token step for  $job$ 
26:    $s(job) \leftarrow s(job) + 1$ 
27:   if  $job$  finishes then
28:     release KV state and remove  $job$ 
29:   else if  $job$  exhausts its local quantum then
30:     demote  $job$  to a lower-priority queue
31:   end if
32: end for
33: evict low-priority non-imminent requests according to the KV eviction policy
34: return  $B^*$ 
```

Placeholder for Decode Scheduling Figure

Recommended drawing: a two-part figure.
 Left: requests move across $Q_1, \dots, Q_n$ as attained service increases; starving requests are promoted.
 Right: KV state transitions remote $\rightarrow$ prefetching $\rightarrow$ resident, followed by possible cold-state eviction back to remote.
 At the bottom, show that an iteration batch is admitted only for requests whose priority is high enough and whose KV can be activated under memory and prefetch budgets.
 Annotate three benefits: responsiveness for short jobs, bounded starvation, and controllable memory footprint.

Fig. 6. Illustration of decode scheduling in PhaseServe. Requests are organized by attained service and urgency, admitted subject to KV and prefetch budgets, and updated at iteration boundaries to balance responsiveness, fairness, and throughput.

E. KV-State Manager

A key design choice in PhaseServe is to elevate KV cache to a first-class managed system resource. The KV-state manager tracks where each request state resides and coordinates prefetch, transfer, and eviction with the local schedulers.

1) KV Residency States: For each active request, the KV-state manager tracks one of four residency states:

- resident: the KV state is already in the target GPU memory and can be used immediately;
- staged: metadata and address mapping are installed, but the state is not yet fully ready for execution;
- prefetching: the state is currently being transferred or restored;
- remote: the state is stored in host memory or another device and is not immediately executable.

This explicit state machine decouples logical request scheduling from physical state placement. The scheduler reasons about readiness through residency states rather than issuing ad hoc swap operations.

2) Proactive Prefetch and Cold-State Eviction: When a request becomes likely to run in the near future, the decode engine issues a prefetch request before the iteration in which the request is expected to be admitted. Conversely, when memory pressure rises, PhaseServe evicts low-priority, non-imminent requests using a cold-state policy that considers both future urgency and reactivation cost.

Let the KV size of request r be $S_{\text{KV}}(r)$ and the effective transfer bandwidth be B . The transfer time is

$$T_{\text{swap}}(r) = \frac{S_{\text{KV}}(r)}{B}. \quad (13)$$

PhaseServe attempts to hide this cost by overlapping prefetch with ongoing decode computation whenever possible. The exposed latency is therefore only the portion of transfer time that cannot be hidden behind useful work.

3) Prefill-to-Decode State Handoff: When a prefill engine completes a request, it materializes the initial KV state and registers the handoff with the KV-state manager. The actual handoff to the selected decode engine is then scheduled according to the routing decision and the current transfer budget. Because state movement is explicitly modeled, PhaseServe can trade off locality and balance: a decode engine with slightly higher queue pressure may still be preferred if the state-transfer path is significantly cheaper.

F. Implementation Notes

PhaseServe is implemented on top of a phase-disaggregated serving runtime. The global controller executes asynchronously with respect to local engine execution and maintains only lightweight summaries instead of per-iteration centralized state. Prefill and decode engines therefore make local decisions independently within the policy envelope set by the controller at each control epoch. This separation keeps the control path lightweight and prevents the controller from becoming a runtime bottleneck.

The design intentionally relies only on low-cost signals such as input length, current context length, queue age, KV footprint, and transfer readiness. This choice avoids the brittleness of exact latency prediction while still exposing enough structure for effective scheduling and control.

G. Discussion

The design of PhaseServe rests on a simple but important principle: after prefill and decode are disaggregated, the dominant bottleneck often shifts from inter-phase interference to intra-phase heterogeneity and cross-phase imbalance. Prefill benefits from known-size, utility-aware batch construction; decode benefits from service-aware iterative scheduling; and the system as a whole benefits from an explicit control plane that reasons about load, state, and topology jointly.

Compared with naive phase-disaggregated serving, PhaseServe makes phase-aware control explicit at three levels. At the local scheduling level, it uses different scheduling abstractions for known-size prefill and unknown-size decode. At the state-management level, it turns KV cache into an explicitly managed resource that enables concurrency-aware decode scheduling. At the cross-phase coordination level, it continuously regulates routing and effective phase balance under non-stationary workloads. Together, these mechanisms allow PhaseServe to simultaneously improve TTFT, TPOT, and overall serving efficiency under realistic heterogeneous workloads.

IV. Implementation

PhaseServe is implemented on top of the open-source DistServe [15] framework. Our implementation mainly extends DistServe’s scheduler to support phase-specialized scheduling, including cost-aware request routing, length-aware prefill scheduling with capacity-aware batching, and MLFQ-based decode scheduling with proactive KV-cache management. The system uses NCCL [20] for tensor and pipeline parallel communication, and each serving instance is realized as a distributed inference engine built from multiple Ray [21] actors. Prefill and decode remain fully disaggregated as in DistServe, while a lightweight profiler provides cost estimates that guide scheduling and routing decisions. KV-cache state is managed in block form to support decode-side queue transitions and cache movement with low overhead. Overall, PhaseServe preserves the underlying distributed serving stack of DistServe while adding the mechanisms needed to realize phase-specialized scheduling.

V. Evaluation

We evaluate PhaseServe to answer the following questions:

- Does PhaseServe improve TTFT and TPOT over representative unified and phase-disaggregated baselines?
- Do the proposed prefill and decode schedulers contribute complementary benefits?
- Under the tested deployment setting, does PhaseServe improve SLO attainment under heterogeneous workloads?

We compare PhaseServe against DistServe [15], a representative phase-disaggregated serving system, and vLLM [9], a widely used unified serving engine. We emphasize

TABLE I
Statistics of prompt and output token lengths for the evaluated datasets

Dataset	Prompt Tokens			Output Tokens		
	Avg	Median	P90	Avg	Median	P90
ShareGPT	768.2	695	1556	195.9	87.0	518.0
HumanEval	131.3	117.0	225.7	53.8	45.5	106.8

that our goal is to evaluate the effect of phase-specialized scheduling under a fixed deployment setting, rather than to exhaustively benchmark every recent serving runtime.

A. Experimental Setup

Testbed. We deployed PhaseServe in a node consisting of 8 NVIDIA PCIe H20-96 GB GPUs. Each GPU is connected to the other via an NVLink bridge with 900 GB/s bidirectional bandwidth. In addition, the CPU is a 128 vCPU AMD EPYC 9K84 96-core processor with 1200 GB of memory. Our system uses a fully-connected NVLink topology (NV18), where all eight GPUs are directly connected via point-to-point NVLink links. Although the machine spans two NUMA domains, GPU-GPU communication never traverses PCIe or the Root Complex, resulting in a uniform NVLink bandwidth fabric.

Models. To evaluate PhaseServe, we employ the OPT [3] series in our experiments, as they are widely regarded as representative in the research community. The OPT family comprises models of various sizes, ranging from 125 million to 175 billion parameters. We use the OPT-6.7B and OPT-66B models for our evaluation. All model parameters in our experiments are configured with FP16 precision.

Workloads. We evaluate the capabilities of PhaseServe using two representative workloads that correspond to common LLM inference scenarios. The dataset statistics used in our experiments are summarized in Table I. For chat-oriented inference, we adopt the ShareGPT [18] dataset, which contains real-world multi-turn conversations collected from ChatGPT users. The dataset features diverse prompt and response lengths, making it well-suited for stress-testing prefill and decode scheduling under highly variable input conditions. For code-generation workloads, we employ the HumanEval [6] benchmark, a widely used dataset for evaluating functional code generation. Compared with dialogue datasets, HumanEval requests typically contain short problem descriptions but require models to generate longer and structurally constrained outputs, providing a complementary workload pattern for testing decode-stage performance and KV-cache management.

Placement Strategies. Considering the model, workload characteristics, and SLO attainment targets, DistServe decides the allocation of prefill and decoding instances through simulation. PhaseServe follows a similar approach to define its parallelism strategy, with the instance placement used in our experiments summarized in Table II.

Metrics. We report TTFT and TPOT as the primary latency metrics, and include both median and P90 values

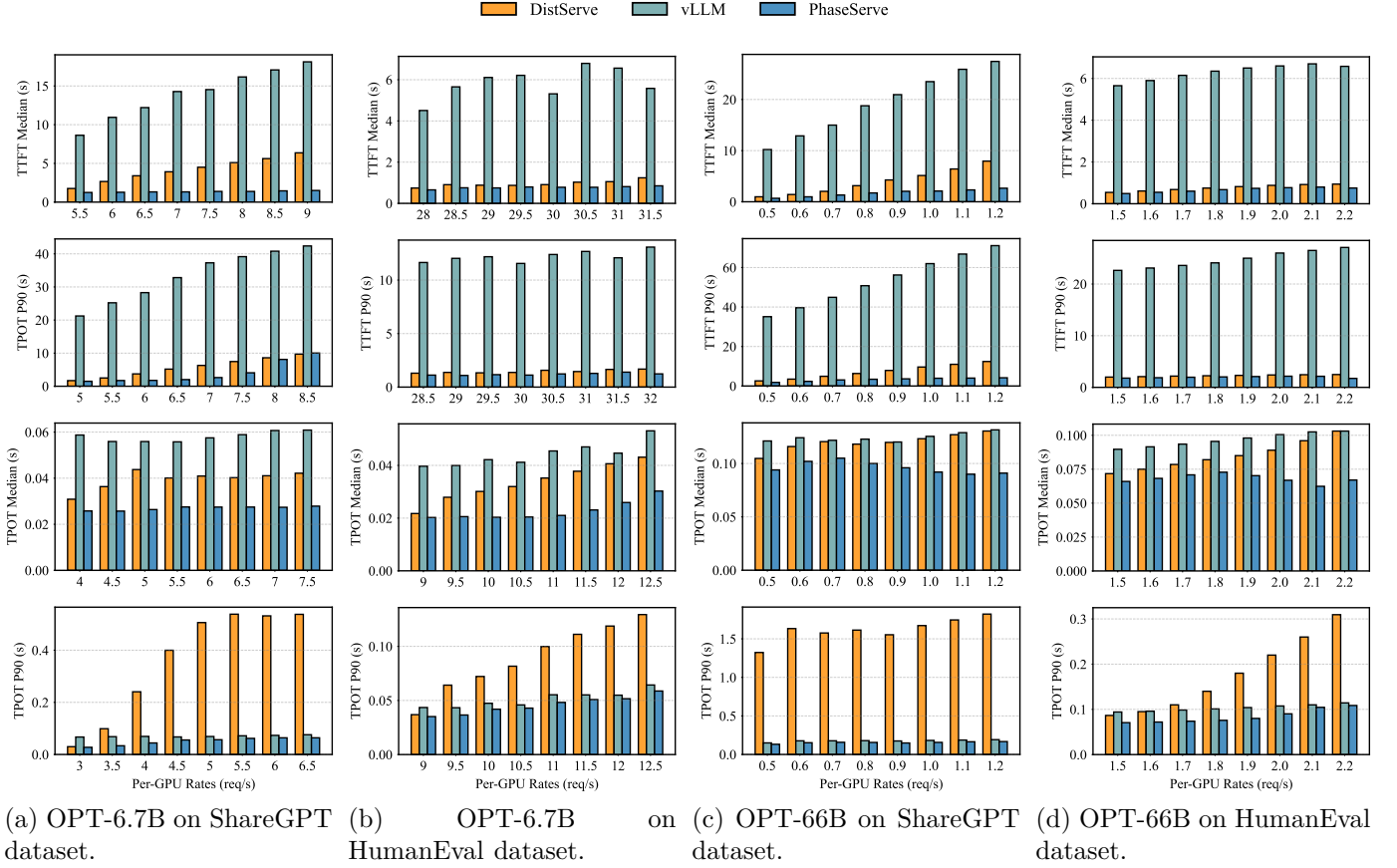


Fig. 7. End-to-end latency of PhaseServe compared with DistServe and vLLM.

TABLE II
Placement strategies (TP = tensor parallelism, PP = pipeline parallelism)

Model	Prefill Placement	Decode Placement
OPT-6.7B	TP-2, PP-1	TP-2, PP-1
OPT-66B	TP-2, PP-2	TP-2, PP-2

to capture typical and tail behavior. We additionally report SLO attainment to measure how often requests satisfy target service thresholds under increasing load.

B. End-to-End Performance

Overall trend. Across both workloads and both model scales, PhaseServe consistently reduces TTFT and TPOT relative to the baselines over the tested load ranges. The improvement is especially pronounced in tail latency, indicating that phase-specialized scheduling is particularly beneficial under heterogeneous workloads and increasing contention.

ShareGPT. Figures 7a and 7c show the results on ShareGPT. This workload stresses both prefill scheduling, due to long and highly variable prompt lengths, and decode scheduling, due to diverse response lengths. Under increasing request rate, PhaseServe maintains substantially lower TTFT than both DistServe and vLLM. For example, on OPT-6.7B, when the request rate increases from 5.5 to 9 req/s, PhaseServe keeps median TTFT

around 1.2–1.5 s, while DistServe reaches 4–6 s and vLLM exceeds 10 s. This corresponds to approximately $2.5\times$ speedup over DistServe and up to $5\times$ over vLLM. Tail TTFT shows a similar trend.

PhaseServe also improves TPOT on ShareGPT. Median TPOT remains around 0.025–0.028 s, corresponding to about $1.4\times$ – $1.6\times$ speedup over DistServe and $2\times$ – $2.5\times$ over vLLM. P90 TPOT remains below 0.06 s over the tested range, whereas DistServe exhibits substantially larger tail latency at high load. These improvements are consistent with the design of PhaseServe: length-aware prefill scheduling reduces queueing and head-of-line blocking for short requests, while attained-service-aware decode scheduling improves responsiveness during iterative generation.

Overall, on the ShareGPT dataset, PhaseServe significantly outperforms both baselines in first-token response and continuous generation, showing that phase-specialized scheduling is effective under long-context and highly variable dialogue workloads.

HumanEval. Figures 7b and 7d show the results on HumanEval. Compared with ShareGPT, this workload has shorter prompts and relatively lighter prefill pressure, but still exposes decode-side variability. PhaseServe continues to outperform both baselines on TTFT and TPOT. On OPT-6.7B, when the request rate is 28–31.5 req/s, median TTFT remains at 0.65–0.85 s, compared with 0.9–1.2 s for

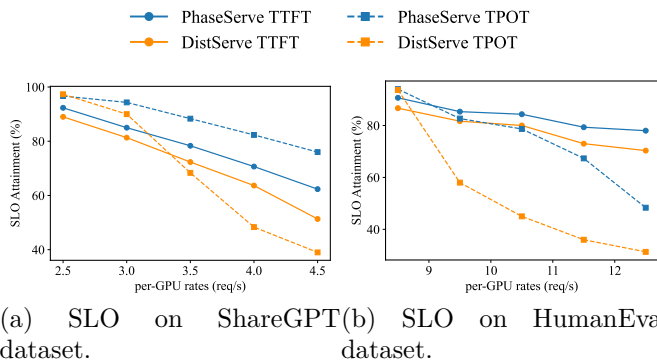


Fig. 8. SLO attainment rates on OPT-6.7B.

TABLE III
Absolute SLO thresholds used in the evaluation.

Model	Dataset	TTFT (s)	TPOT (s)
OPT-6.7B	ShareGPT	0.4	0.03
OPT-6.7B	HumanEval	0.1	0.03

DistServe and 4–6 s for vLLM. This corresponds to about $1.5\times$ speedup over DistServe and up to $9\times$ over vLLM. P90 TTFT follows the same pattern, remaining at 1.1–1.4 s while vLLM exceeds 12 s.

For TPOT, PhaseServe maintains median TPOT latency around 0.02–0.03 s, lower than DistServe (0.03–0.04 s) and vLLM (0.04–0.05 s), corresponding to about $1.5\times$ speedup over DistServe and about $2\times$ over vLLM. P90 TPOT remains around 0.06 s across the tested range.

The HumanEval results suggest that the benefit of PhaseServe is not limited to long-prompt dialogue workloads. Even when prefill heterogeneity is less severe, decode-side scheduling still provides measurable latency gains.

Interpretation. Taken together, these results support the main claim of this paper: once prefill and decode are disaggregated, the remaining latency bottlenecks are largely intra-phase rather than inter-phase. PhaseServe improves performance by matching scheduling policy to phase characteristics: known-size scheduling for prefill and unknown-size scheduling for decode. The effect is most visible in tail latency, where heterogeneous workloads would otherwise amplify queueing and straggler effects.

C. SLO Attainment

The SLO attainment results are shown in Figure 8, and the thresholds are summarized in Table III. Across the tested rates, PhaseServe consistently achieves higher SLO attainment than DistServe. On ShareGPT, at a per-GPU request rate of 4.5 req/s, PhaseServe improves TTFT and TPOT SLO satisfaction by $1.21\times$ and $1.94\times$, respectively.

These results complement the latency curves by showing that the gains of PhaseServe are not limited to average-case behavior. Instead, they translate into a higher fraction of requests meeting service targets under load. This is particularly important for production settings, where predictable latency often matters more than peak throughput alone.

At the same time, the absolute SLO values also indicate that the tested configuration eventually becomes overloaded as request rate increases. We therefore interpret SLO attainment primarily as a comparative measure across systems under the same deployment setting, rather than as a claim that all workloads satisfy strict production targets at the highest tested load.

D. Ablation Studies

In this section, we study the individual effects of the prefill and decode schedulers in PhaseServe. We construct two variants: PhaseServe-no-prefill-scheduler, which replaces the prefill-side length-aware scheduling and capacity-aware batching with a DistServe-style FCFS policy and fixed batch size; and PhaseServe-no-decode-scheduler, which replaces the MLFQ-based decode scheduler with a DistServe-style FCFS policy in the decoding phase. We evaluate these variants on the OPT-6.7B model.

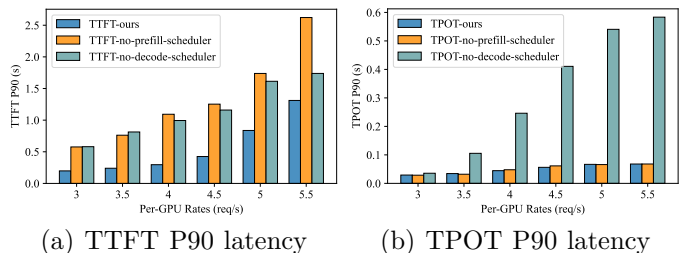


Fig. 9. Ablation Studies

As shown in Figure 9, disabling the prefill scheduler results in a noticeable increase in TTFT, particularly at higher request rates. This indicates that the prefill-side design of PhaseServe—namely length-aware request ordering and capacity-aware batch construction—is effective in reducing head-of-line blocking and queueing delay under heterogeneous prompt lengths. In contrast, disabling the decode scheduler primarily degrades TPOT, leading to higher token-level latency under load. This result is consistent with the role of the MLFQ-based scheduler in prioritizing lightly served requests and improving responsiveness during iterative decoding.

PhaseServe with both schedulers enabled consistently achieves the best overall latency profile across the tested per-GPU rates. These results suggest that the two schedulers provide complementary benefits: the prefill scheduler mainly improves request-entry latency, while the decode scheduler mainly improves generation-stage responsiveness.

Overall, the evaluation consistently shows that PhaseServe improves TTFT, TPOT, and SLO attainment under the tested deployment setting. These results support our central claim that, after prefill and decode are disaggregated, intra-phase scheduling remains a dominant source of latency, and that phase-specialized scheduling is effective in reducing this bottleneck.

VI. Related Work

Inference Serving Optimization. Recent research on LLM inference serving spans general-purpose systems such as Triton [22], as well as LLM-specific solutions like Orca [8], vLLM [9], and SARATHI [10]. These systems improve throughput and hardware utilization through techniques such as iterative or continuous batching, paged-attention for KV-cache management, and chunked-prefill with piggybacked decoding. However, most prior work colocates prefill and decoding on the same instance, which can lead to interference, pipeline bubbles, and under-utilized resources. To address this, disaggregated prefill-decode architectures have been proposed by DistServe [15], Splitwise [16], DéjàVu [17], separating stages across instances to enable independent scheduling and better resource utilization. PhaseServe builds on this idea but focuses on practical scheduling and batch management within each phase—length-aware scheduling and capacity-aware batch construction in prefill, and MLFQ with KV-cache management in decode.

Model Parallelism. LLMs with hundreds of billions of parameters require parallel execution across multiple GPUs. Prior work [14], [23]–[28] has explored various strategies for model partitioning and scheduling. Pope [25] propose analytical models for selecting multi-dimensional partitioning strategies tailored to TPU slices. HeteGen [26] introduces heterogeneous parallelism across CPUs and GPUs, leveraging asynchronous overlap to mitigate I/O bottlenecks. ExeGPT [27] optimizes batch size and tensor parallelism degree based on input/output distributions to maximize throughput under latency constraints. Helix [28] formulates model partitioning across heterogeneous GPUs and network links as a max-flow problem, capturing both compute and communication heterogeneity. PhaseServe can integrate these model-parallelism techniques into its distributed inference engine, enabling flexible and efficient scaling of LLM workloads.

Kernel-Level Optimizations. Efficient attention computation is critical for LLM inference. Techniques such as FlashAttention [29] and FlashAttention-2 [30] reduce memory access overhead between GPU high-bandwidth memory and on-chip SRAM, improving compute efficiency for long-context sequences. Concurrent approaches like POD-Attention [31] optimize GPU utilization for hybrid batches, enabling concurrent execution of prefill and decode tasks. Other works explore sparse attention to reduce memory footprint, though they may compromise generation accuracy. PhaseServe is orthogonal to these kernel-level optimizations: our work focuses on scheduling and resource management at the serving-system level, and can be naturally combined with optimized attention kernels to further improve throughput and efficiency.

VII. Conclusion and Future Work

In this work, we presented PhaseServe, a phase-disaggregated LLM serving system built around the observation that prefill and decoding exhibit fundamentally

different scheduling characteristics. Prefill requests have known but highly heterogeneous execution costs, whereas decode requests reveal their total cost only incrementally during generation. Motivated by this asymmetry, PhaseServe applies phase-specialized scheduling to the two stages: length-aware scheduling with capacity-aware batch construction for prefill, and MLFQ-based attained-service-aware scheduling with proactive KV-cache management for decoding.

Our results show that, under the tested deployment setting, this design consistently improves both TTFT and TPOT over representative unified and phase-disaggregated baselines. These findings suggest that, even after prefill and decode are separated, intra-phase scheduling remains a major source of latency and resource inefficiency, and that aligning scheduling policy with phase characteristics is an effective way to address this bottleneck.

There are several promising directions for future work. First, our current study focuses on a single-node NVLink-connected environment; extending PhaseServe to multi-node deployments would enable a deeper understanding of how phase-specialized scheduling interacts with cross-node KV transfer and network heterogeneity. Second, PhaseServe currently assumes a fixed prefill/decode deployment configuration. Incorporating dynamic resource reallocation and heterogeneous hardware support would further improve adaptability under changing workload conditions.

More broadly, we believe that treating prefill and decoding as distinct scheduling regimes provides a useful perspective for future LLM serving systems, especially as deployment environments and workload diversity continue to grow.

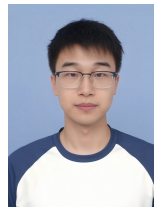
Acknowledgment

This work was supported by the open research fund of Pengcheng Laboratory under Grant 2025KF1A0020, and the Joint Funds of the National Natural Science Foundation of China under Grant U22A2036.

References

- [1] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar et al., “Llama: Open and efficient foundation language models,” arXiv preprint arXiv:2302.13971, 2023.
- [2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., “Language models are few-shot learners,” vol. 33, 2020, pp. 1877–1901.
- [3] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin et al., “Opt: Open pre-trained transformer language models,” arXiv preprint arXiv:2205.01068, 2022.
- [4] OpenAI, “Introducing chatgpt,” 2022. [Online]. Available: <https://openai.com/blog/chatgpt>
- [5] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez et al., “Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality,” 2023.
- [6] M. Chen, “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.

- [7] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez et al., “Code llama: Open foundation models for code,” arXiv preprint arXiv:2308.12950, 2023.
- [8] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for {Transformer-Based} generative models,” in 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), 2022, pp. 521–538.
- [9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in Proceedings of the 29th symposium on operating systems principles, 2023, pp. 611–626.
- [10] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, “Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills,” arXiv preprint arXiv:2308.16369, 2023.
- [11] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, “Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve},” in 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024, pp. 117–134.
- [12] NVIDIA, “Tensorrt-llm: A tensorrt toolbox for optimized large language model inference,” 2024. [Online]. Available: <https://github.com/NVIDIA/TensorRT-LLM>
- [13] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” in International Conference on Machine Learning. PMLR, 2023, pp. 31 094–31 116.
- [14] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez et al., “{AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving,” pp. 663–679, 2023.
- [15] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024, pp. 193–210.
- [16] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA). IEEE, 2024, pp. 118–132.
- [17] F. Strati, S. McAllister, A. Phanishayee, J. Tarnawski, and A. Klimovic, “Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving,” in Proceedings of the 41st International Conference on Machine Learning, 2024, pp. 46 745–46 771.
- [18] ShareGPT Team, “Sharegpt,” 2023. [Online]. Available: <https://sharegpt.com/>
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” vol. 30, 2017.
- [20] NVIDIA, “Nvidia collective communications library (nccl),” 2023. [Online]. Available: <https://developer.nvidia.com/nccl>
- [21] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan et al., “Ray: A distributed framework for emerging {AI} applications,” in 13th USENIX symposium on operating systems design and implementation (OSDI 18), 2018, pp. 561–577.
- [22] NVIDIA, “Triton inference server: An optimized cloud and edge inferencing solution,” 2019.
- [23] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” arXiv preprint arXiv:1909.08053, 2019.
- [24] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing et al., “Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning,” in 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), 2022, pp. 559–578.
- [25] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, “Efficiently scaling transformer inference,” Proceedings of machine learning and systems, vol. 5, pp. 606–624, 2023.
- [26] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, “Hetegen: Efficient heterogeneous parallel inference for large language models on resource-constrained devices,” Proceedings of Machine Learning and Systems, vol. 6, pp. 162–172, 2024.
- [27] H. Oh, K. Kim, J. Kim, S. Kim, J. Lee, D.-s. Chang, and J. Seo, “Exegpt: Constraint-aware resource scheduling for llm inference,” in Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, 2024, pp. 369–384.
- [28] Y. Mei, Y. Zhuang, X. Miao, J. Yang, Z. Jia, and R. Vinayak, “Helix: Distributed serving of large language models via max-flow on heterogeneous gpus,” arXiv preprint arXiv:2406.01566, 2024.
- [29] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” vol. 35, 2022, pp. 16 344–16 359.
- [30] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” 2023.
- [31] A. K. Kamath, R. Prabhu, J. Mohan, S. Peter, R. Ramjee, and A. Panwar, “Pod-attention: Unlocking full prefill-decode overlap for faster llm inference,” pp. 897–912, 2025.



Jie Dai obtained his Bachelor's degree in Information Security from Harbin Institute of Technology, China, in 2024. He is currently pursuing his Master's degree at the School of Computer Science and Technology at Harbin Institute of Technology. His research interest lies in large language model inference optimization.



Meng Hao received the BS and Ph.D. degrees in computer science and engineering from Harbin Institute of Technology, China, in 2014 and 2021 respectively. He is currently an associate professor in the Faculty of Computing, Harbin Institute of Technology. His research interests include high-performance computing, performance modeling, and parallel optimization.



Weizhe Zhang (Senior Member, IEEE) received the BEng, MEng, and PhD degrees of engineering in computer science and technology from the Harbin Institute of Technology in 1999, 2001, and 2006 respectively. He is currently a professor with the School of Computer Science and Technology, Harbin Institute of Technology, China, and the director of the Cyberspace Security Research Center, Peng Cheng Laboratory, Shenzhen, China. He has authored or coauthored more than 100

academic papers in journals, books, and conference proceedings. His research interests include parallel computing, distributed computing, cloud and grid computing, and computer network.



Jiayi Li is currently an undergraduate student at the School of Computer Science and Technology, Harbin Institute of Technology, China, and is expected to pursue a Master's degree starting in September 2026. His research focuses on high-performance computing and artificial intelligence, with a particular focus on scheduling optimization for large language model inference systems.



Hongwei Yang is a research associate of the Network Security Center in the School of Cyberspace Science, Harbin Institute of Technology, China. He received the DEng degree in cyberspace science from the Harbin Institute of Technology. His research interests include graph mining, privacy-preserving computing, network security, and big data security.



Hui He received the B.Eng, M.Eng, and Ph.D. degrees from the Harbin Institute of Technology, Harbin, China, in 1997, 1999, and 2006, respectively, all in computer science and technology. She is a Professor in the School of Cyberspace Science, Harbin Institute of Technology. Her current research interests include distributed computing, the Internet of Things, and big data analysis.



Desheng Wang received the Ph.D. degree in Cyberspace Security from Harbin Institute of Technology, Harbin, China, in 2022. He is currently an assistant professor in the School of Computer Science and Technology at Harbin Institute of Technology, Shenzhen, China. His research interests include cloud computing, edge computing, and cyberspace security.